進捗確認タブ UI設計書

メタデータ

- **画面ID**: ProgressCheckTab
- **画面名**: 進捗確認タブ
- **ファイルパス**: `frontend/src/components/ExperimentWorker/ProgressCheckTab.tsx`
- **利用権限**: 実験作業者（experiment_worker）
- **作成日**: 2026-01-19
- **最終更新日**: 2026-03-11

---

1. 概要

1.1 画面の目的

実験作業者が自分に割り当てられたタスクの進捗状況を確認するためのタブ画面です。メソッド単位でタスクをグループ化し、各タスクのステータスと実行可能なアクションを表示します。

1.2 主要機能

- メソッド単位でのタスク一覧表示（タブ形式）
- タスクステータスの確認
- タスク操作（開始、指示確認、実験記録、問題報告）
- メソッドステータスの可視化（バッジ表示）

1.3 アクセス権限

- **対象ロール**: 実験作業者（experiment_worker）
- **必要条件**: ログイン済み、有効なセッション

---

2. 画面構成

2.1 画面イメージ



2.2 レイアウト構造

```
┌──────────────────────────────────────────┐
│ ローディング中（loading=true）              │
│ ┌──────────────────────────────────────┐ │
│ │ Card                                 │ │
│ │  [スピナー]                           │ │
│ │  タスク一覧を読み込んでいます...          │ │
│ └──────────────────────────────────────┘ │
└──────────────────────────────────────────┘

┌──────────────────────────────────────────┐
│ プロジェクト未選択時                        │
│ ┌──────────────────────────────────────┐ │
│ │ Card                                 │ │
│ │  [CheckSquareOutlined アイコン (48px)]  │ │
│ │  進捗確認                             │ │
│ │  プロジェクトを選択してください           │ │
│ └──────────────────────────────────────┘ │
└──────────────────────────────────────────┘

┌──────────────────────────────────────────┐
│ メソッドなし時                             │
│ ┌──────────────────────────────────────┐ │
│ │ Card                                 │ │
│ │  [CheckSquareOutlined アイコン (48px)]  │ │
│ │  メソッドが見つかりません                 │ │
│ │  このプロジェクトにはメソッドが登録されて │ │
│ │  いません                             │ │
│ └──────────────────────────────────────┘ │
└──────────────────────────────────────────┘

プロジェクト選択後:
┌──────────────────────────────────────────┐
│ Card                                     │
│ ┌──────────────────────────────────────┐ │
│ │ メソッドタブ（Tabs type="card"）        │ │
│ │ ┌──┬──┬──┐                          │ │
```

```
| | |メソ|メソ|メソ|                          | |
| | |ド1|ド2|ド3| (各タブにバッジ表示)       | |
| | └──┴──┴──┘                              | |
| |                                          | |
| | ┌──────────────────────────────────┐ | |
| | | ProgressCheckTable                | | |
| | | - タスク一覧テーブル              | | |
| | | - アクションボタン                | | |
| | └──────────────────────────────────┘ | |
| └──────────────────────────────────────┘ |
└──────────────────────────────────────────┘

モーダル:
- IssueReportModal（問題報告）
- IssueManagementModal（問題確認 - 参照専用）
```

2.3 使用コンポーネント

Ant Design コンポーネント

- `Card` - メインコンテナ
- `Tabs` - メソッド切り替えタブ（type="card"）
- `Badge` - メソッドステータス表示
- `Space` - 要素の間隔制御
- `Typography.Title` - 見出し
- `Typography.Text` - テキスト表示
- `theme` - デザイントークン取得

カスタムコンポーネント

- `ProgressCheckTable` - タスク進捗テーブル
- `IssueReportModal` - 問題報告モーダル
- `IssueManagementModal` - 問題確認モーダル（参照専用）

アイコン

- `CheckSquareOutlined` - チェックボックスアイコン（空状態）

- `RightOutlined` - 右矢印（将来拡張用）

---

3. コントロール一覧

3.1 入力項目

項目名	タイプ	必須	初期値	説明
selectedProject	表示のみ	○	props経由	選択中のプロジェクトID（親から受け取り）
selectedMethodId	Tab選択	○	自動選択	選択中のメソッドID（最初のメソッドを自動選択）

3.2 タブ

各メソッドタブには以下の情報を表示：

- メソッド名（14px、font-weight: 500）
- ステータスバッジ + タスク数（12px、colorTextSecondary）

ステータスバッジの種類

ステータス	Badge status	ラベル	表示条件
未着手	default	未着手	全タスクが未着手
作業中	processing	作業中	開始済み or 作業中のタスクがある
完了	success	完了	全タスクが完了
エラー	error	エラー	強制終了タスクがある

3.3 テーブル内アクション

`ProgressCheckTable` コンポーネント内で以下のアクションボタンを提供：

アクション	アクションキー	説明	遷移先
開始	`start`	タスクを開始（API連携・楽観的ロック対応）	-

指示確認	`instruction`	指示確認タブに遷移	InstructionConfirmationTab（タブ2）
実験記録	`record`	実験記録タブに遷移	ExperimentRecordTab（タブ3）
問題報告	`issue`	問題報告モーダルを開く	IssueReportModal
問題確認	`issueCheck`	問題確認モーダルを開く（参照専用）	IssueManagementModal
強制終了	`forceTerminate`	タスクを強制終了（API連携・楽観的ロック対応）	-

---

4. イベント

4.1 タブ選択イベント

イベント名	トリガー	処理内容
onSelectMethod	メソッドタブクリック	選択されたメソッドIDを親コンポーネントに通知

4.2 タスクアクションイベント

イベント名	トリガー	処理内容
handleTaskAction	タスクアクションボタンクリック	アクション種別に応じた処理を実行

アクション別の処理内容

```
switch (action) {
  case 'start':
    // タスク開始処理（API連携・楽観的ロック対応）
    await updateTaskStatus(
      selectedProjectName,
      task.methodId,
      task.taskNo,
      'in_progress',
      undefined,      // comment（オプショナル）
      task.updatedAt  // currentUpdatedAt（楽観的ロック用）
    );
    message.success('タスクを開始しました');
    // タスク一覧を再取得して最新の状態を反映
```

```
      await refreshTasks();
      break;

    case 'instruction':
      // 指示確認タブへ遷移（メソッドとタスクを選択状態にする）
      if (onSelectTaskAndTab) {
        onSelectTaskAndTab(taskId, '2'); // 指示確認タブ
        message.info('指示確認タブに移動します');
      } else {
        // フォールバック: メソッドのみ選択
        if (task.methodId) onSelectMethod(task.methodId);
        if (onTabChange) onTabChange('2');
      }
      break;

    case 'record':
      // 実験記録タブへ遷移（タスクを選択状態にする）
      if (onSelectTaskAndTab) {
        onSelectTaskAndTab(taskId, '3'); // 実験記録タブ
        message.info('実験記録タブに移動します');
      } else if (onTabChange) {
        onTabChange('3');
      }
      break;

    case 'issue':
      // 問題報告モーダルを開く
      handleOpenIssueReport(task);
      break;

    case 'issueCheck':
      // 問題確認モーダルを開く（参照専用）
      handleOpenIssueCheck(task);
      break;

    case 'forceTerminate':
      // 強制終了処理（API連携・楽観的ロック対応）
      await updateTaskStatus(
        selectedProjectName,
        task.methodId,
        task.taskNo,
        'forced_terminated',
        undefined,      // comment（オプショナル）
        task.updatedAt  // currentUpdatedAt（楽観的ロック用）
      );
      message.success('タスクを強制終了しました');
      // タスク一覧を再取得して最新の状態を反映
      await refreshTasks();
      break;
  }
```

エラーハンドリング（`start` / `forceTerminate` 共通）：

- `403`：「このタスクを操作する権限がありません」
- `404`：「タスクが見つかりません」
- `409`：「このタスクは他のユーザーによって更新されました」 → タスク一覧を再取得
- その他:「タスクの更新中にエラーが発生しました」

4.3 モーダルイベント

イベント名	トリガー	処理内容
handleOpenIssueReport	問題報告ボタンクリック	選択したタスクを設定し、IssueReportModalを開く
handleCloseIssueReport	問題報告モーダルを閉じる	モーダルを非表示、選択タスクをクリア
handleOpenIssueCheck	問題確認ボタンクリック	選択したタスクを設定し、IssueManagementModalを開く（参照専用）
handleCloseIssueCheck	問題確認モーダルを閉じる	モーダルを非表示、選択タスクをクリア

---

5. データフロー

5.1 Props（親から受け取るデータ）

```
interface ProgressCheckTabProps {
  selectedProject: string | null;              // 選択中のプロジェクトID
  selectedProjectName: string;                 // 選択中のプロジェクト名（API呼び出しに使用）
  onProjectChange: (projectId: string) => void; // プロジェクト変更ハンドラ
  selectedMethodId: string | null;             // 選択中のメソッドID
  onSelectMethod: (methodId: string) => void;  // メソッド選択ハンドラ
  onTabChange?: (tabKey: string) => void;      // タブ変更ハンドラ
  onSelectTaskAndTab?: (taskId: string, tabKey: string) => void; // タスク選択＋タブ遷移
  readOnly?: boolean;                          // trueのとき強制終了ボタンを無効化（オーバーレイ閲
```

```
覧モード用）
}
```

5.2 State（内部状態）

```
// API連携の状態管理
const [loading, setLoading] = useState<boolean>(false);          // データ取得中フラグ
const [methods, setMethods] = useState<Method[]>([]);            // メソッド一覧
const [tasks, setTasks] = useState<Task[]>([]);                  // タスク一覧（全メソッド分）
const [projectId, setProjectId] = useState<string | null>(null); // APIから取得したプロジェクトID

// プロジェクトステータス（暫定: APIから取得できるまでnull）
const projectStatus: ProjectStatus | null = null;

// 自動メソッド選択の制御
const prevProjectRef = useRef<string | null>(null);              // 前回のプロジェクトID（変更検知用）
const [needsAutoSelect, setNeedsAutoSelect] = useState<boolean>(false); // 自動選択フラグ
```

注意: プロジェクト変更時は `needsAutoSelect` フラグを使った2段階の自動選択ロジックで制御します。 これにより、プロジェクト切り替え時に確実に最初のメソッドが自動選択されます。

5.3 算出データ

```
// メソッドタブ（useMemo）
const methodTabs = useMemo(() => {
  return methods.map((method) => {
    // このメソッドに紐づくタスクのみをフィルタリング
    const methodTasks = tasks.filter(task => task.methodId === method.methodId);

    // メソッドのステータスを取得
    const methodStatus = getMethodStatus(methodTasks);

    return {
      key: method.methodId,
      label: (タブラベルJSX),
      children: (ProgressCheckTable)
    };
```

```
  });
}, [methods, tasks, projectStatus, handleTaskAction, token, getMethodStatus]);
```

注意: `methods` と `tasks` はAPIから取得した値をStateとして保持しており、`useMemo` ではなく `useState` で管理します。

5.4 useEffect

```
// ① プロジェクト選択時にAPIでタスク一覧を取得
useEffect(() => {
  const fetchWorkerTasks = async () => {
    if (!selectedProject || !selectedProjectName) {
      setMethods([]);
      setTasks([]);
      setProjectId(null);
      return;
    }
    setLoading(true);
    try {
      const result = await getWorkerTasks(selectedProjectName);
      if (result.success && result.data) {
        const apiData = result.data;
        setProjectId(apiData.project_id);

        // メソッド一覧を変換（UI用の型に変換）
        const methodsData: Method[] = apiData.methods.map((method: WorkerMethod) => ({
          methodId: method.method_id,
          methodName: method.method_name,
          experimentOrder: method.experiment_order,
          description: '',        // APIには存在しないためデフォルト値
          category: 'experiment', // APIには存在しないためデフォルト値
          transitions: [],        // APIには存在しないためデフォルト値
        }));

        // 全タスクを変換（全メソッド分をフラット化）
        const allTasks: Task[] = apiData.methods.flatMap((method: WorkerMethod) =>
          method.tasks.map((task: WorkerTask) => ({
            taskId: `${apiData.project_id}-${method.method_id}-${task.task_no}`,
            taskNo: task.task_no.toString(),
            taskName: task.task_no.toString(), // task_nameカラム削除のためtask_noを代用
            methodId: method.method_id,
            methodName: method.method_name,
            status: task.task_status as Task['status'],
            assignedUserId: task.experiment_user_id,
            assignedUserName: task.experiment_user_name,
```

```
            createdAt: task.created_at,
            updatedAt: task.updated_at,
            projectId: apiData.project_id,
            sampleNumber: '',
            notes: '',
            progressRate: parseFloat(task.progress_rate) || 0,
            transitionStatuses: [],
            progressReport: '',
            recordNumber: '',
          }))
        );

        setMethods(methodsData);
        setTasks(allTasks);

        // API取得完了後、最初のメソッドを自動選択
        if (methodsData.length > 0) {
          const sortedMethods = [...methodsData].sort((a, b) => a.experimentOrder -
b.experimentOrder);
          onSelectMethod(sortedMethods[0].methodId);
        }
        message.success('タスク一覧を取得しました');
      } else {
        message.error('タスク一覧の取得に失敗しました');
        setMethods([]);
        setTasks([]);
      }
    } catch (error) {
      message.error('タスク一覧の取得中にエラーが発生しました');
      setMethods([]);
      setTasks([]);
    } finally {
      setLoading(false);
    }
  };
  fetchWorkerTasks();
}, [selectedProject, selectedProjectName]);

// ② プロジェクト変更を検知して自動選択フラグをセット
useEffect(() => {
  if (selectedProject !== prevProjectRef.current) {
    prevProjectRef.current = selectedProject;
    setNeedsAutoSelect(true);
  }
}, [selectedProject]);

// ③ メソッドが更新されたら自動選択
// - needsAutoSelect=true（プロジェクト変更後）: 先頭メソッドを強制選択
// - selectedMethodId=null（初回表示時など）: 通常の自動選択
useEffect(() => {
```

```
  if (methods.length > 0 && needsAutoSelect) {
    const sortedMethods = [...methods].sort((a, b) => a.experimentOrder - b.experimentOrder);
    onSelectMethod(sortedMethods[0].methodId);
    setNeedsAutoSelect(false);
  } else if (methods.length > 0 && !selectedMethodId && !needsAutoSelect) {
    const sortedMethods = [...methods].sort((a, b) => a.experimentOrder - b.experimentOrder);
    onSelectMethod(sortedMethods[0].methodId);
  }
}, [methods, needsAutoSelect, selectedMethodId, onSelectMethod]);
```

5.5 使用するAPI

API	メソッド	用途	実装状況	サービス関数
タスク一覧取得API	GET	プロジェクト内の全タスク取得	✅ 完了	`getWorkerTasks(projectName)`
タスクステータス更新API	PATCH	タスクステータスの更新（楽観的ロック対応）	✅ 完了	`updateTaskStatus(projectName, methodId, taskNo, status, comment, currentUpdatedAt)`
問題報告API	POST	問題報告の送信	✅ 完了	-
コメント投稿API	POST	問題へのコメント投稿	✅ 完了	-
問題一覧取得API	GET	問題一覧取得	✅ 完了	-
問題詳細取得API	GET	問題詳細とコメント一覧取得	✅ 完了	-
問題ステータス変更API	PATCH	問題ステータスの変更	✅ 完了	-

タスクステータス更新時のエラーハンドリング:

- `403`：権限なしエラー
- `404`：タスク未存在エラー
- `409`：楽観的ロック競合エラー（他ユーザーが先に更新） → タスク一覧を再取得して最新状態を反映

5.6 データソース

- `getWorkerTasks` - プロジェクト名を指定してメソッド・タスク一覧を取得（`@/services/projectApiService`）
- `updateTaskStatus` - タスクステータス更新（楽観的ロック対応）（`@/services/projectApiService`）
- `WorkerTasksResponse`, `WorkerMethod`, `WorkerTask` - APIレスポンスの型定義（`@/services/projectApiService`）

---

6. エラーチェック

6.1 バリデーション

項目	ルール	エラーメッセージ
プロジェクト選択	必須	「プロジェクトを選択してください」（空状態表示）
タスク選択	必須（アクション実行時）	「プロジェクトを選択してください」（message.warning）
タスク情報	存在確認（アクション実行時）	「タスク情報が見つかりません」（message.error）

6.2 プロジェクトステータスによる制限

アクション実行時に`ProgressCheckTable`コンポーネント内でステータスチェックを実施

---

7. 画面遷移

7.1 遷移元

- ExperimentWorkerMenu（実験作業メニュー）から「進捗確認」タブを選択

7.2 遷移先

遷移先	トリガー	遷移方法
指示確認タブ（タブ2）	「指示確認」ボタンクリック	タブ切り替え（onTabChange または onSelectTaskAndTab）
実験記録タブ（タブ3）	「実験記録」ボタンクリック	タブ切り替え（onTabChange または onSelectTaskAndTab）

7.3 モーダル遷移

```
ProgressCheckTab
  ├→ IssueReportModal（問題報告）
  │    └→ 閉じる
  └→ IssueManagementModal（問題確認 - 参照専用）
       └→ 閉じる
```

---

8. レスポンシブ対応

8.1 ブレークポイント

サイズ	幅	対応
Mobile	< 768px	タブは縦スクロール、テーブルは横スクロール
Tablet	768px - 1024px	標準レイアウト
Desktop	> 1024px	標準レイアウト

8.2 タブラベルのレスポンシブ

- Mobile: タブラベルは2行表示（メソッド名 + ステータス）
- Desktop: 同様に2行表示

---

9. パフォーマンス考慮事項

9.1 最適化

- `methods` ・ `tasks` ・ `projectId` ・ `loading` は `useState` で管理（APIから取得したデータ）
- `methodTabs` は `useMemo` でメモ化
- `handleTaskAction` は `useCallback` でメモ化
- `getMethodStatus` は `useCallback` でメモ化

9.2 データ量

- メソッド数: 最大10件程度を想定
- タスク数: 1メソッドあたり最大50件程度を想定

---

10. 備考

10.1 特記事項

1. **自動メソッド選択**
   - プロジェクト選択後、最初のメソッドを自動選択
   - `useEffect` で実装
2. **メソッドステータスの算出ロジック**

```
const getMethodStatus = (methodTasks: Task[]) => {
  if (methodTasks.length === 0) return { status: 'default', label: '未着手' };

  // 強制終了タスクがあればエラー
  if (methodTasks.some(t => t.status === 'forced_terminated')) {
    return { status: 'error', label: 'エラー' };
  }

  // 全タスクが完了していれば完了
  if (methodTasks.every(t => t.status === 'completed')) {
    return { status: 'success', label: '完了' };
  }

  // 作業中のタスクがあれば作業中（'in_progress'のみ対象）
  if (methodTasks.some(t => t.status === 'in_progress')) {
```

```
    return { status: 'processing', label: '作業中' };
  }

  return { status: 'default', label: '未着手' };
};
```

3. **タスクIDとメソッドIDの管理**
   - 各タスクに `projectId` と `methodId` を付与
   - アクション実行時にタスクから `methodId` を取得

4. **参照専用の問題確認モーダル**
   - `IssueManagementModal` を `viewMode={true}` で開く
   - 編集不可、参照のみ

10.2 将来の拡張

1. タスクフィルタリング機能（ステータス、期限）
2. タスクソート機能
3. 承認依頼機能（プロジェクト単位）
4. リアルタイム進捗更新（WebSocket）

10.3 関連ドキュメント

- ProgressCheckTable UI設計書
- IssueManagementModal UI設計書

---

11. 変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2026-01-19	初版作成	Cline

1.1.0	2026-03-11	ソースコード差分に基づく仕様書修正（Props追加・API連携実装反映・IssueReportModal追加・アクション処理更新・ローディング状態追加）	Cline